



# Insurance Premium — Test Results

Automated Test Documentation for Model Governance

MKM Research Labs

Version 1.0 — 03-April-2026

David K Kelly

**CONFIDENTIAL — PROPRIETARY**

## **Legal Notice**

### **PROPRIETARY AND CONFIDENTIAL**

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs (“MKM”).

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an “as is” basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

All rights reserved. © 2021–2026 MKM Research Labs.

Governed by the laws of the United Kingdom.

# 1 Test Results — Insurance Premium (MKM-IP-001)

Tests run on **03 April 2026 at 20:15:57**.

Table 1: Test Results Summary — Insurance Premium (MKM-IP-001)

|             |       |
|-------------|-------|
| Total Tests | 36    |
| Passed      | 36    |
| Failed      | 0     |
| Skipped     | 0     |
| Duration    | 0.00s |

Table 2: Detailed Test Results — Insurance Premium

| Test Description                  | Acceptance Criteria                                                         | Result | Time   |
|-----------------------------------|-----------------------------------------------------------------------------|--------|--------|
| All factors applied               | <code>abs(result - round(expected, 2)) &lt; 0.01</code>                     | Pass   | 0.000s |
| Basic calculation                 | <code>result == 1250.0</code>                                               | Pass   | 0.000s |
| Claims factor increases premium   | <code>with_claims &gt; base</code>                                          | Pass   | 0.000s |
| Clamped to max                    | <code>result == MAX_PREMIUM</code>                                          | Pass   | 0.000s |
| Clamped to min                    | <code>result == MIN_PREMIUM</code>                                          | Pass   | 0.000s |
| Contents premium added            | <code>with_contents == base + 500.0</code>                                  | Pass   | 0.000s |
| High flood risk increases premium | <code>high_flood &gt; low_flood</code>                                      | Pass   | 0.000s |
| Returns float                     | <code>isinstance(result, float)</code>                                      | Pass   | 0.000s |
| Security factor reduces premium   | <code>with_security &lt; without</code>                                     | Pass   | 0.000s |
| 100 years is heritage             | <code>get_age_premium.band(1925, current_year=2025) == 'heritage'</code>    | Pass   | 0.000s |
| 10 years is modern                | <code>get_age_premium.band(2015, current_year=2025) == 'modern'</code>      | Pass   | 0.000s |
| 30 years is established           | <code>get_age_premium.band(1995, current_year=2025) == 'established'</code> | Pass   | 0.000s |
| Default current year              | <code>result in AGE_PREMIUM_FACTORS</code>                                  | Pass   | 0.000s |
| Established                       | <code>get_age_premium.band(1960, current_year=2025) == 'established'</code> | Pass   | 0.000s |
| Exactly 29 years modern           | <code>get_age_premium.band(1996, current_year=2025) == 'modern'</code>      | Pass   | 0.000s |
| Exactly 99 years established      | <code>get_age_premium.band(1926, current_year=2025) == 'established'</code> | Pass   | 0.000s |
| Exactly 9 years new build         | <code>get_age_premium.band(2016, current_year=2025) == 'new_build'</code>   | Pass   | 0.000s |
| Heritage old                      | <code>get_age_premium.band(1850, current_year=2025) == 'heritage'</code>    | Pass   | 0.000s |

| Test Description                      | Acceptance Criteria                                                                                    | Result | Time   |
|---------------------------------------|--------------------------------------------------------------------------------------------------------|--------|--------|
| Modern                                | <code>get_age_premium_band(2000, current_year=2025) == 'modern'</code>                                 | Pass   | 0.000s |
| New build                             | <code>get_age_premium_band(2020, current_year=2025) == 'new.build'</code>                              | Pass   | 0.000s |
| Exactly 100 third band                | <code>(lo, hi) == AREA_PREMIUM_BANDS[2][1]</code>                                                      | Pass   | 0.000s |
| Exactly 50 second band                | <code>(lo, hi) == AREA_PREMIUM_BANDS[1][1]</code>                                                      | Pass   | 0.000s |
| Just below 100 second band            | <code>(lo, hi) == AREA_PREMIUM_BANDS[1][1]</code>                                                      | Pass   | 0.000s |
| Just below 50 first band              | <code>(lo, hi) == AREA_PREMIUM_BANDS[0][1]</code>                                                      | Pass   | 0.000s |
| Large area fourth band                | <code>(lo, hi) == AREA_PREMIUM_BANDS[3][1]</code>                                                      | Pass   | 0.000s |
| Medium area second band               | <code>(lo, hi) == AREA_PREMIUM_BANDS[1][1]</code>                                                      | Pass   | 0.000s |
| Returns tuple                         | <code>isinstance(result, tuple);<br/>len(result) == 2</code>                                           | Pass   | 0.000s |
| Small area first band                 | <code>(lo, hi) == AREA_PREMIUM_BANDS[0][1]</code>                                                      | Pass   | 0.000s |
| Age premium factors all positive      | <code>lo &gt; 0; hi &gt;= lo</code>                                                                    | Pass   | 0.000s |
| Area bands are sorted                 | <code>thresholds == sorted(thresholds)</code>                                                          | Pass   | 0.000s |
| Base rate range                       | <code>lo &gt; 0; hi &gt; lo</code>                                                                     | Pass   | 0.000s |
| Construction type factors present     | <code>'Brick' in CONSTRUCTION_TYPE_FACTORS;<br/>'Timber Frame' in<br/>CONSTRUCTION_TYPE_FACTORS</code> | Pass   | 0.000s |
| Flood risk factors increase with risk | <code>very_high_hi &gt; very_low_lo</code>                                                             | Pass   | 0.000s |
| Flood risk factors present            | <code>set(FLOOD_RISK_PREMIUM_FACTORS.keys())<br/>== expected</code>                                    | Pass   | 0.000s |
| Min max premium bounds                | <code>MIN_PREMIUM == 200;      MAX_PREMIUM ==<br/>20_000</code>                                        | Pass   | 0.000s |
| Property type factors present         | <code>set(PROPERTY_TYPE_PREMIUM_FACTORS.keys())<br/>== expected</code>                                 | Pass   | 0.000s |

## 1.1 Test Document History

| Date        | Version | Description                   | Author   |
|-------------|---------|-------------------------------|----------|
| 03-Apr-2026 | 1.0     | Initial test suite (36 tests) | D. Kelly |